

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 6

End-to-End Study of RNN, LSTM and GRU for Sequence Learning and Video Understanding

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to develop an end-to-end understanding of recurrent sequence learning by implementing and comparing Vanilla RNN, LSTM and GRU models. The study covers temporal sequence classification, training and validation convergence, confusion-matrix analysis, the effect of sequence length, CNN-assisted video understanding, and an encoder–decoder sequence-to-sequence task.

The primary experiment uses raw inertial signals from the UCI Human Activity Recognition Using Smartphones dataset. The workflow prepares temporal sequences, visualizes representative sensor signals, trains the three recurrent architectures under controlled settings, evaluates them using multiple metrics, and then extends the same temporal-learning idea to video features extracted by a frozen CNN.

2. Learning Outcomes

After completing this experiment, the student will be able to:

- represent sequential data using the (N, T, F) format;
- explain Vanilla RNN operation and Backpropagation Through Time;
- explain vanishing and exploding gradient problems;
- implement sequence classification using SimpleRNN, LSTM and GRU;
- compare models using accuracy, macro precision, macro recall and macro F1;
- interpret training/validation curves and confusion matrices;
- explain the spatial feature-extraction role of a CNN in video understanding;
- construct a CNN–LSTM and CNN–GRU temporal video classifier; and
- explain encoder–decoder sequence-to-sequence learning and token versus sequence accuracy.

3. Dataset and Experimental Setup

Primary Dataset: UCI Human Activity Recognition Using Smartphones

The UCI HAR dataset contains six activity classes: WALKING, WALKING_UPSTAIRS, WALKING_DOWNSTAIRS, SITTING, STANDING and LAYING. The raw inertial signals used here contain 128 temporal measurements and nine channels: three body-acceleration axes, three body-gyroscope axes and three total-acceleration axes.

The resulting input representation is

$$X \in \mathbb{R}^{N \times 128 \times 9}.$$

The data were divided into 70% training, 15% validation and 15% testing using a stratified split. Standardization was fitted only on the training set and then applied to validation and test sets.

Dimension	Meaning	Value
N	Number of sequences	Dataset-dependent
T	Time steps	128
F	Sensor channels	9

Controlled recurrent-model settings: all three models used 32 recurrent units and the same classifier and evaluation protocol. This ensures that the observed differences are primarily associated with the recurrent architecture.

4. Overall Experimental Pipeline



For video understanding, the pipeline becomes: video $\rightarrow$ ten uniformly sampled frames $\rightarrow$ MobileNetV2 features $\rightarrow$ a 10×1280 temporal feature sequence $\rightarrow$ LSTM/GRU $\rightarrow$ action class.

5. Preprocessing

1. Load the nine raw inertial signal files.
2. Form 128-step windows with nine channels.
3. Convert activity labels to six integer classes.
4. Create a stratified 70/15/15 train/validation/test split.
5. Fit StandardScaler using training data only.
6. Apply the fitted transformation to validation and test data.
7. Preserve the three-dimensional sequence structure for recurrent processing.

6. Temporal Data Visualization

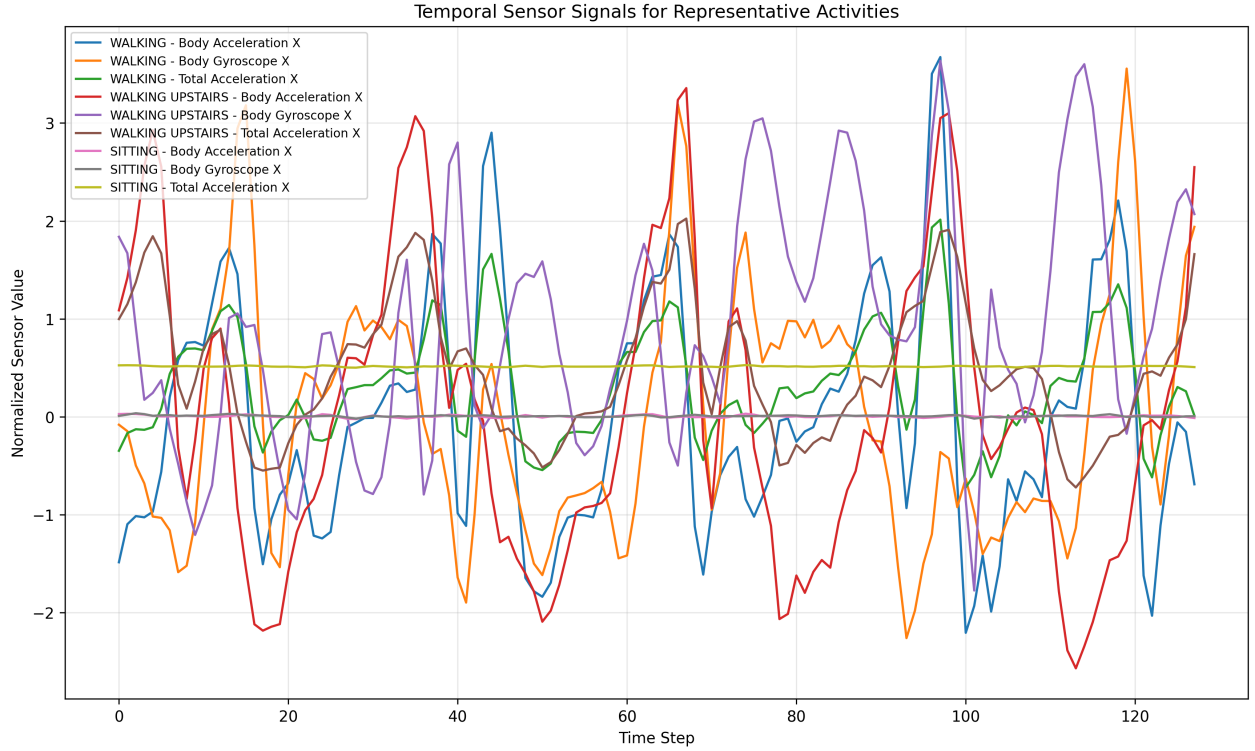


Figure 1: Representative temporal sensor signals for selected activities and channels.

Inference. The plotted signals exhibit activity-dependent temporal patterns. Dynamic walking activities show stronger temporal variation, whereas relatively static activities show more stable patterns. Since the discriminative information is contained not only in individual sensor values but also in their evolution over time, a sequence model is appropriate.

7. Vanilla RNN Architecture

A Vanilla RNN maintains a hidden state h_t that is updated at every time step:

$$h_t = \phi(W_x x_t + W_h h_{t-1} + b_h).$$

The hidden state carries information from earlier time steps. However, repeated multiplication through time can cause gradients to become very small or very large, leading to vanishing or exploding gradients.

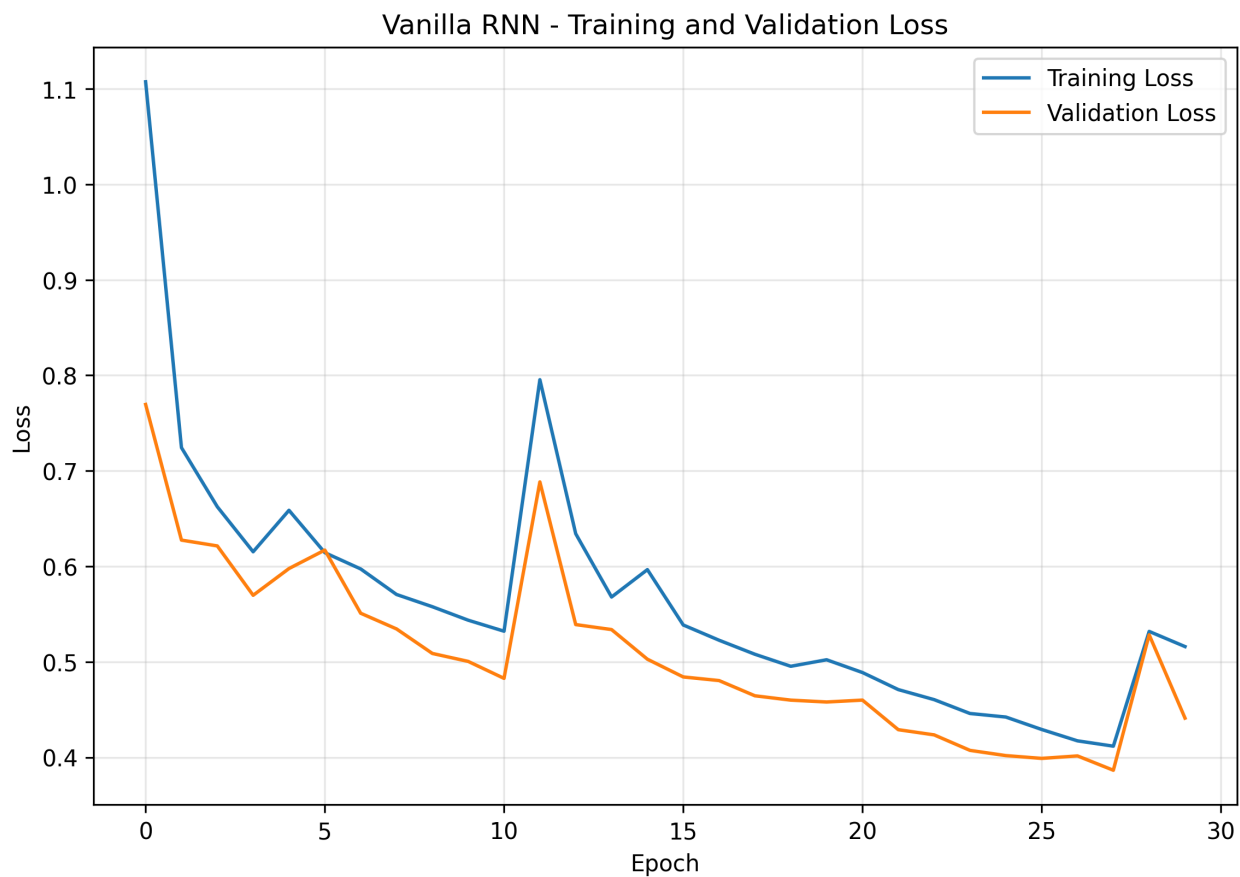


Figure 2: Vanilla RNN training and validation loss.

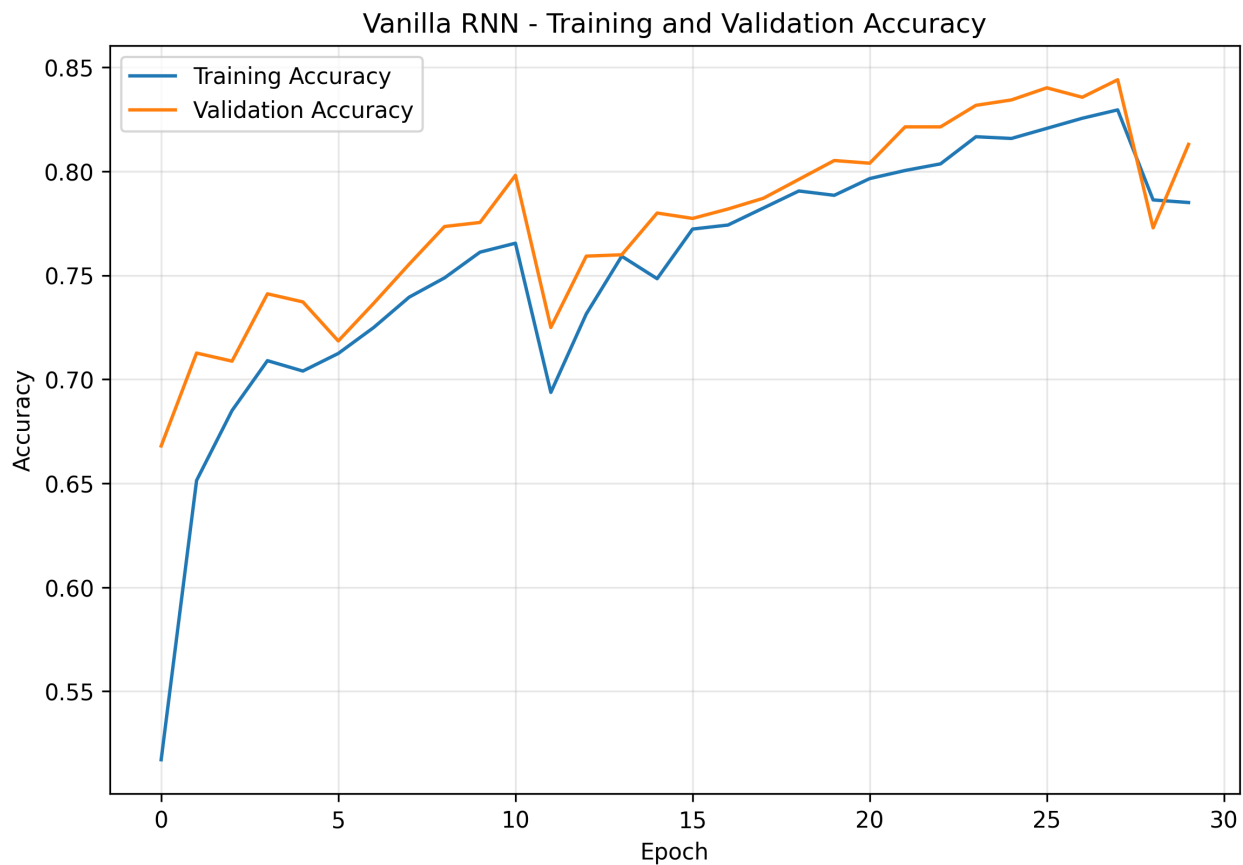


Figure 3: Vanilla RNN training and validation accuracy.

Inference. The RNN learns the activity task, but its curves are more irregular than those of the gated recurrent models. This is consistent with the difficulty of preserving useful information over longer temporal dependencies using a simple recurrent state.

8. LSTM Architecture

LSTM introduces a cell state and gates that regulate information flow. The main gates are the forget, input and output gates. A simplified form is

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f), \quad i_t = \sigma(W_i[h_{t-1}, x_t] + b_i),$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o).$$

The cell state provides a controlled path for information across time.

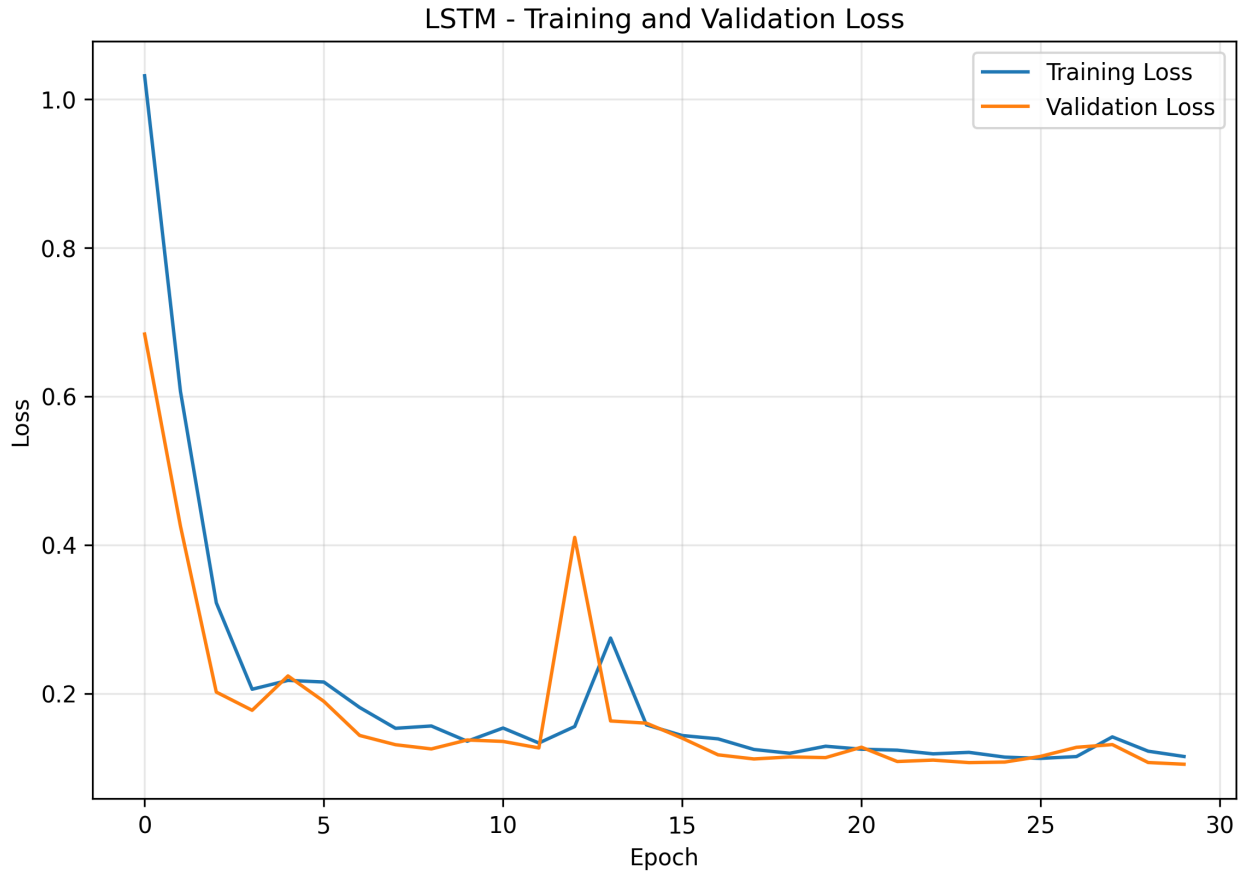


Figure 4: LSTM training and validation loss.

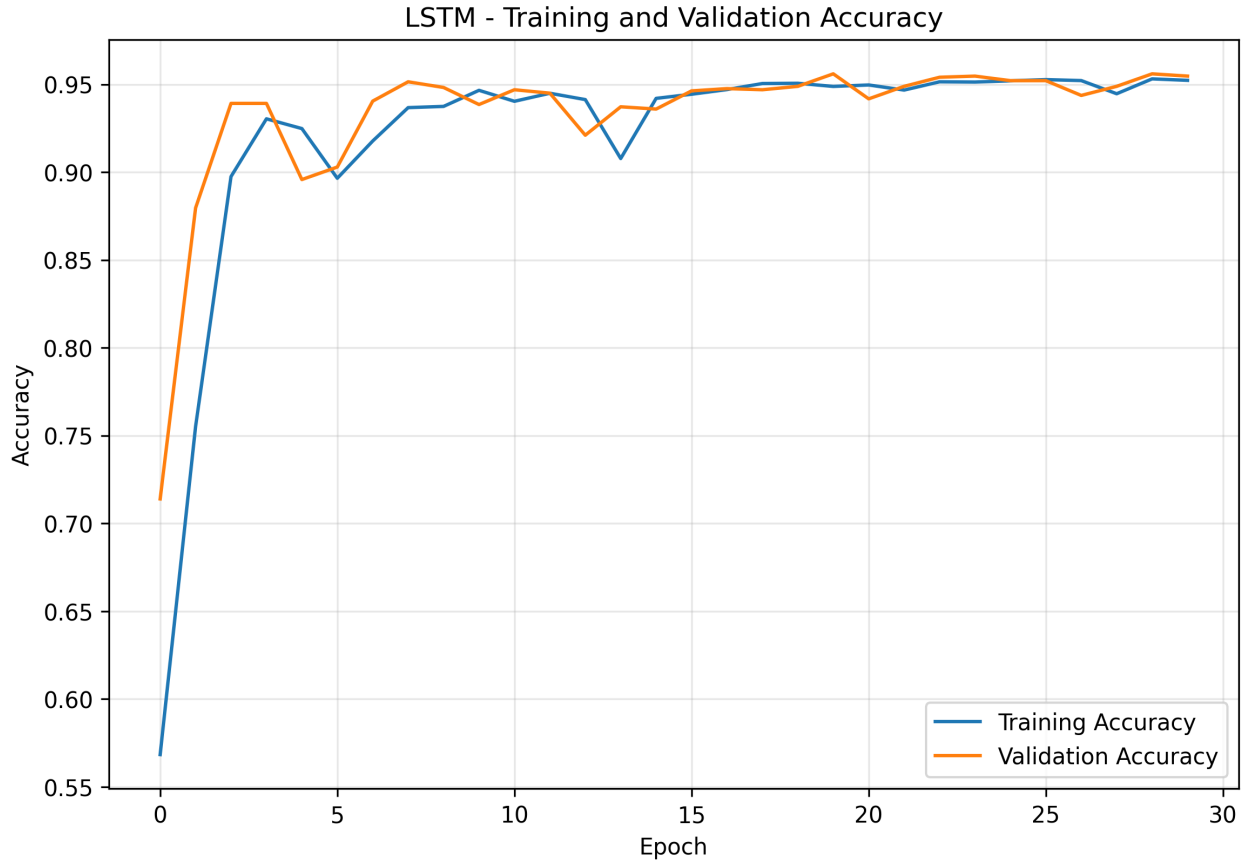


Figure 5: LSTM training and validation accuracy.

Inference. The LSTM curves converge to a high-performance region and remain comparatively stable. The gating mechanism helps preserve useful temporal information and provides more controlled gradient flow.

9. GRU Architecture

GRU uses an update gate and reset gate without a separate cell state. A simplified formulation is

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z), \quad r_t = \sigma(W_r[h_{t-1}, x_t] + b_r).$$

The update gate controls the balance between old and candidate information, while the reset gate controls the influence of previous information when forming a candidate state.

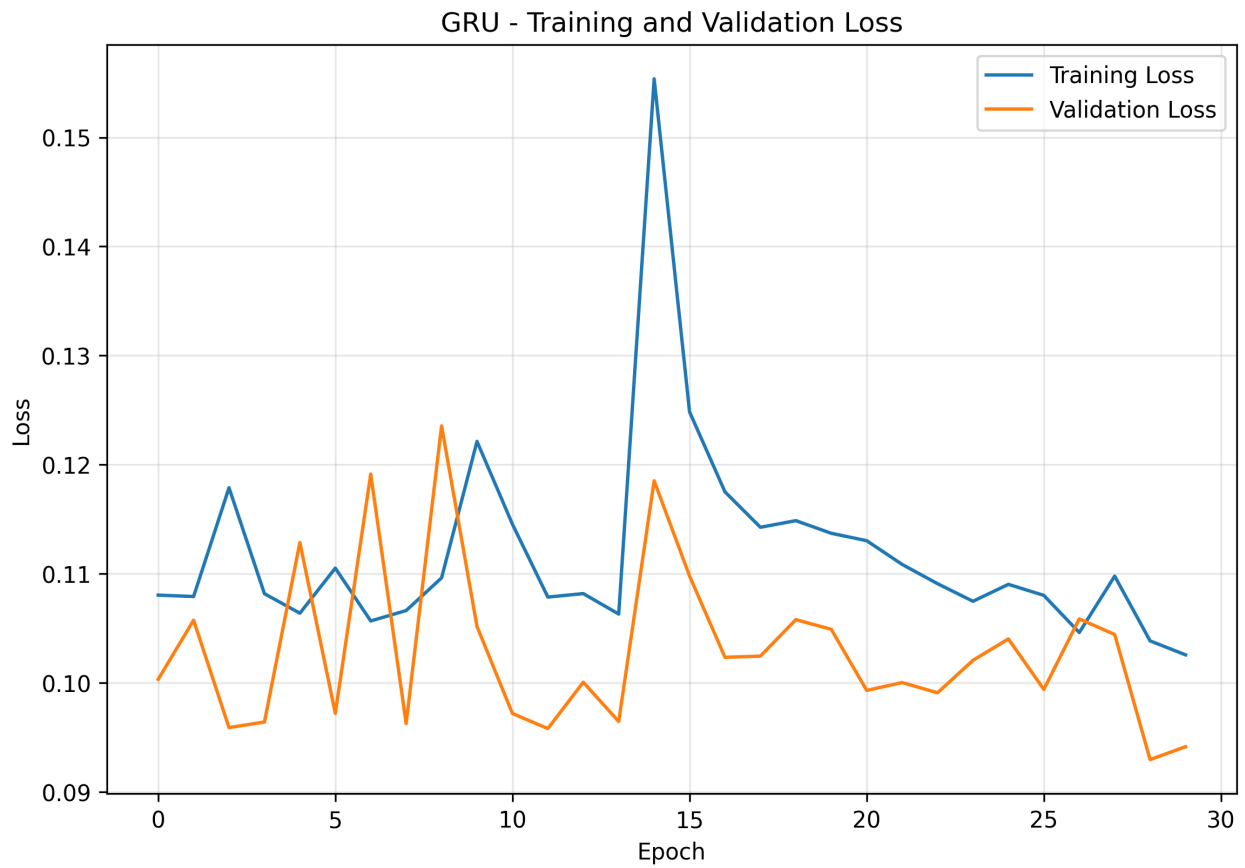


Figure 6: GRU training and validation loss.

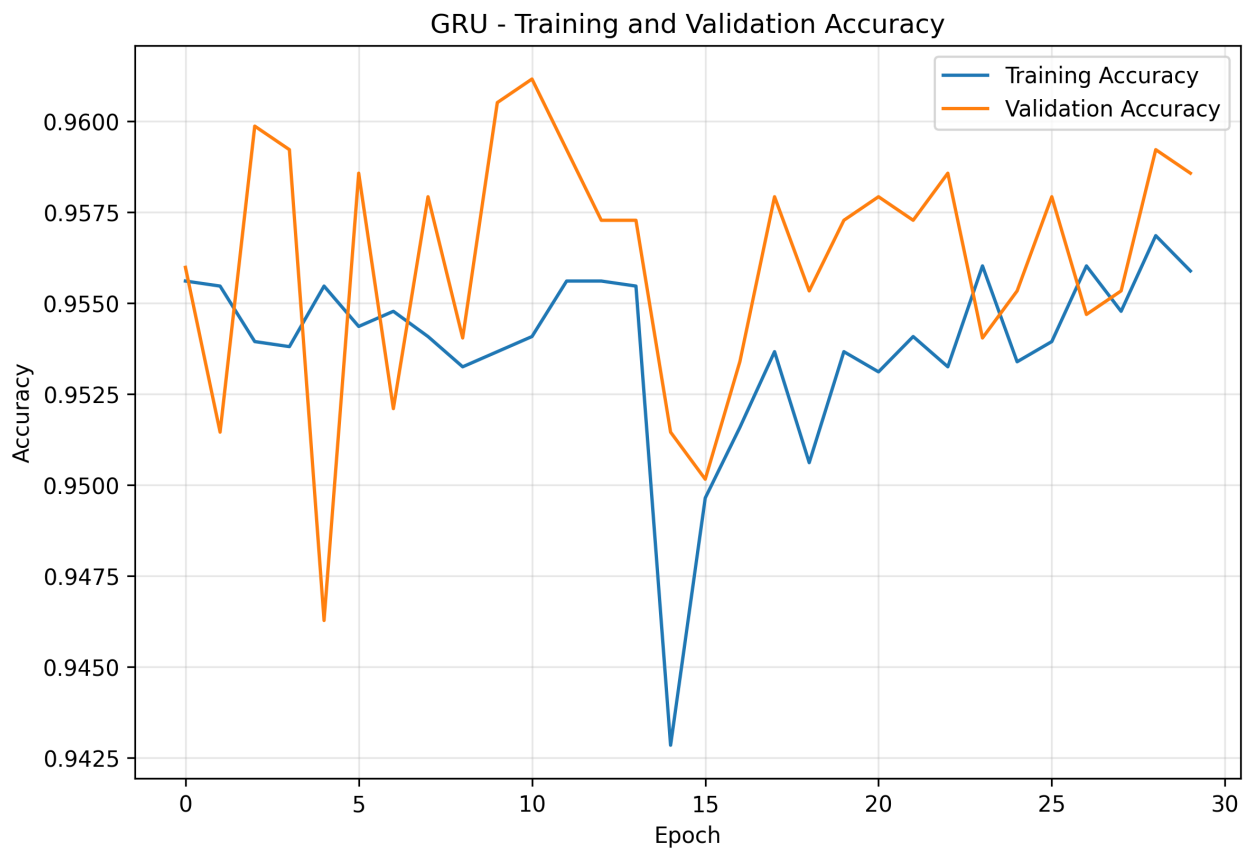


Figure 7: GRU training and validation accuracy.

Inference. The GRU achieves strong accuracy while using fewer parameters than the LSTM in this experiment. Its

simpler gated structure provides an effective compromise between temporal modeling and computational complexity.

10. Quantitative Performance Evaluation

The independent test set was evaluated using accuracy, macro precision, macro recall, macro F1, trainable parameter count and recorded training time.

Model	Accuracy	Precision	Recall	F1	Parameters	Time (s)
Vanilla RNN	80.000%	78.936%	79.112%	78.994%	1,974	78.606
LSTM	94.887%	95.229%	95.221%	95.225%	6,006	78.861
GRU	95.340%	95.661%	95.676%	95.659%	4,758	73.950

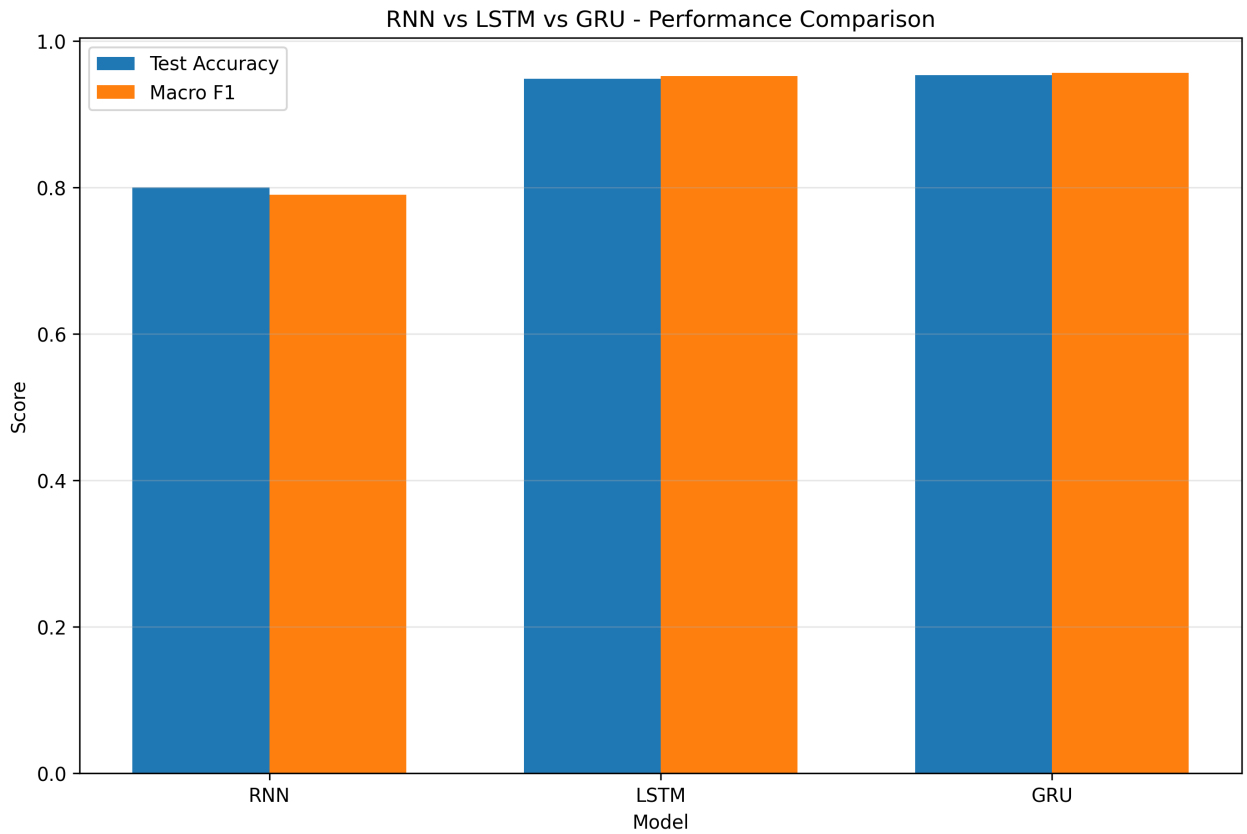


Figure 8: Test performance comparison of Vanilla RNN, LSTM and GRU.

Inference. The gated models substantially outperform the Vanilla RNN on the independent test set. The GRU records the highest test accuracy and macro F1 among the three in this execution, while using fewer parameters and less recorded training time than the LSTM.

11. Confusion Matrix Analysis

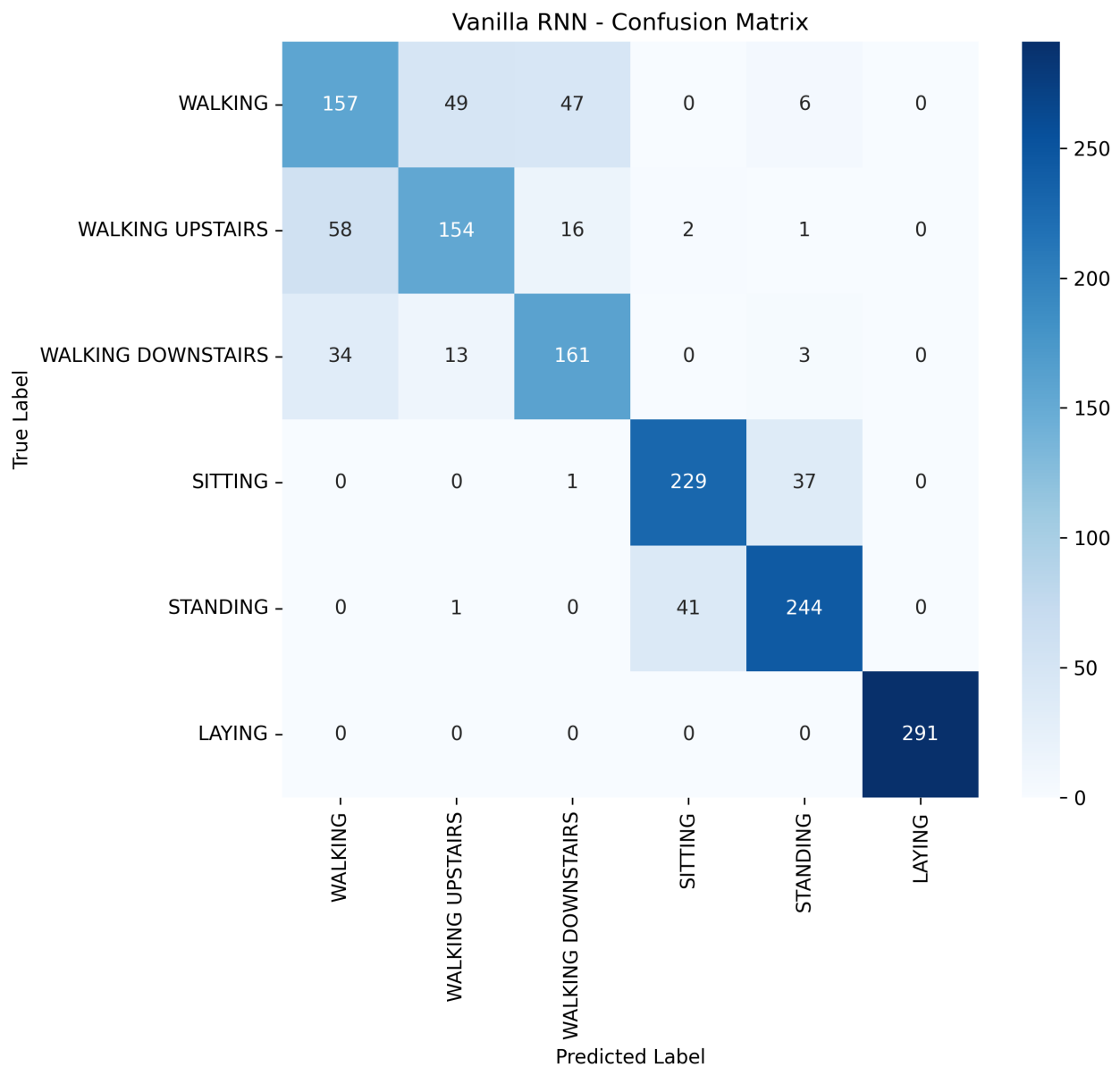


Figure 9: Vanilla RNN confusion matrix.

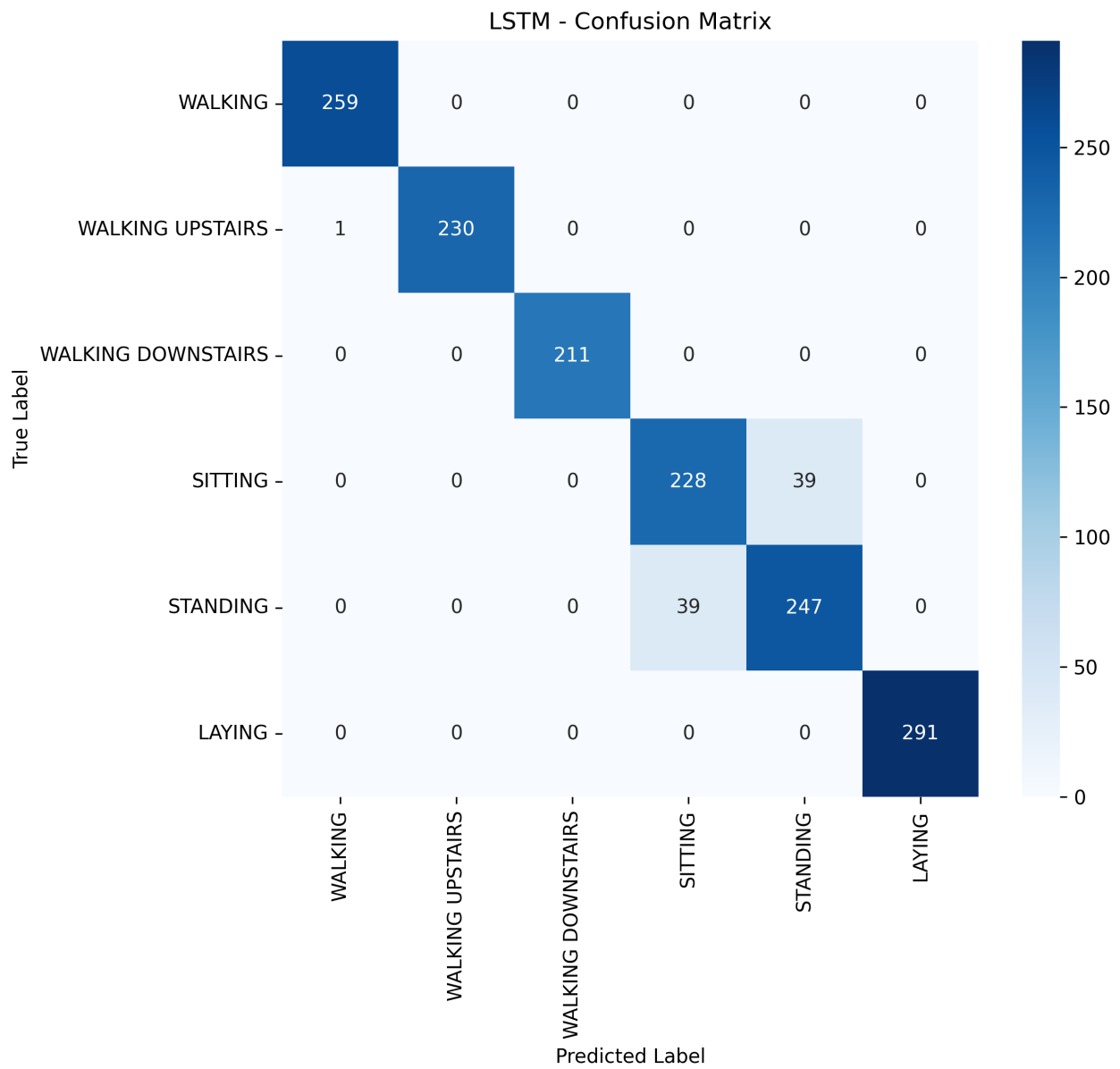


Figure 10: LSTM confusion matrix.

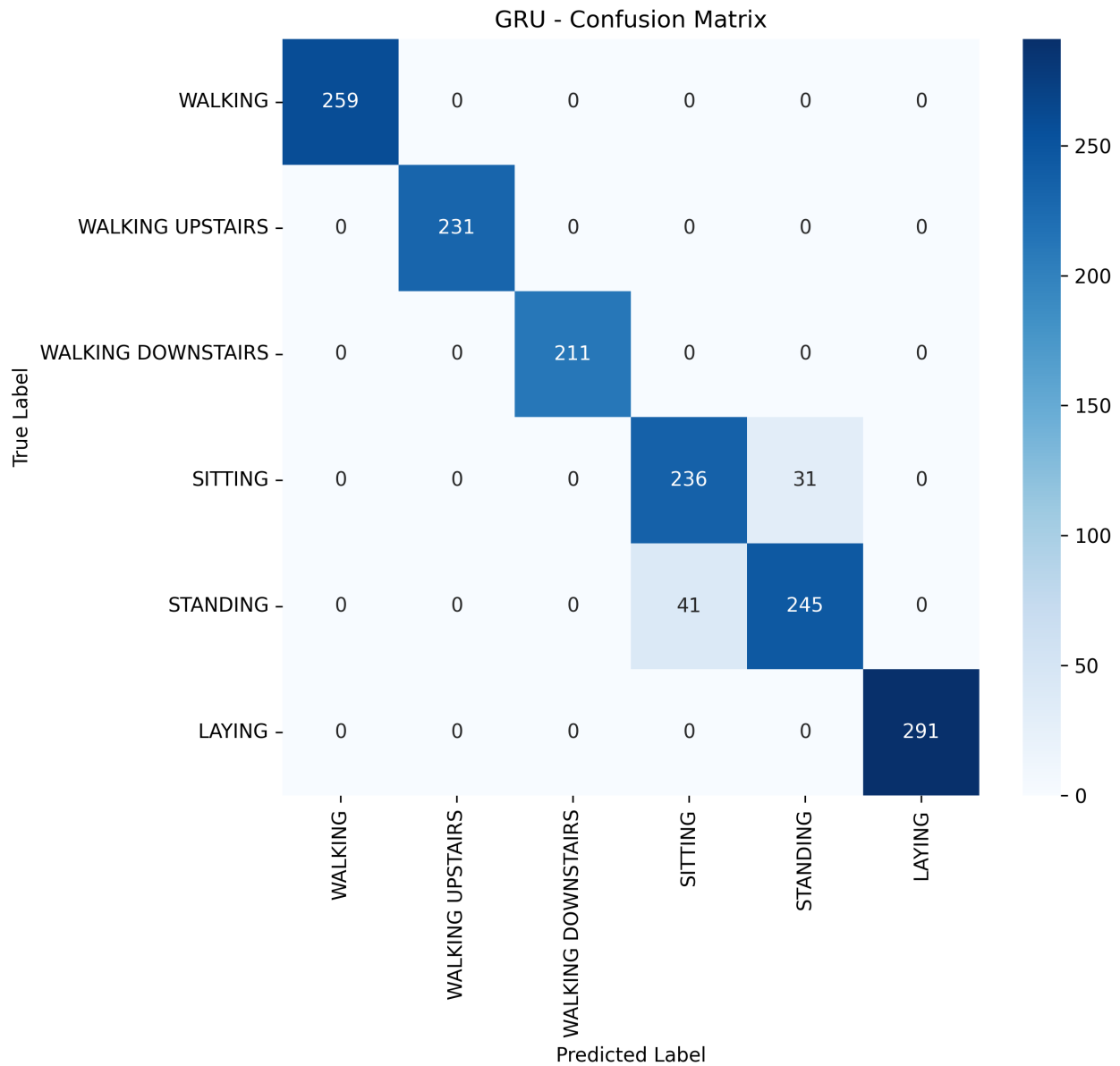


Figure 11: GRU confusion matrix.

The RNN shows appreciable confusion among the three walking-related classes and also between SITTING and STANDING. The LSTM substantially reduces walking-class confusion; its main remaining errors are between SITTING and STANDING. The GRU similarly separates the walking classes strongly, while SITTING and STANDING remain the most persistent confusion pair. LAYING is correctly classified in all displayed cases for all three models.

Inference. Confusion matrices reveal class-specific behavior that a single accuracy number cannot show. Similar activities can have similar temporal sensor patterns, which explains why SITTING and STANDING remain comparatively difficult.

12. RNN vs. LSTM vs. GRU: Parameters and Computational Cost

Model	Trainable Parameters	Training Time (s)	Macro F1
Vanilla RNN	1,974	78.606	78.994%
LSTM	6,006	78.861	95.225%
GRU	4,758	73.950	95.659%

The LSTM has the largest parameter count because its architecture contains multiple gates and a separate cell state. The GRU has fewer parameters because its gating mechanism is simpler. In the recorded runs, the GRU also had the shortest training time of the three.

13. Effect of Sequence Length

The same recurrent comparison was repeated for sequence lengths $T \in \{32, 64, 128\}$.

T	RNN Acc.	RNN F1	LSTM Acc.	LSTM F1	GRU Acc.	GRU F1
32	86.473%	86.172%	94.045%	94.301%	93.981%	94.244%
64	72.816%	70.476%	94.822%	95.114%	94.757%	95.085%
128	69.191%	68.357%	94.498%	94.779%	95.016%	95.339%

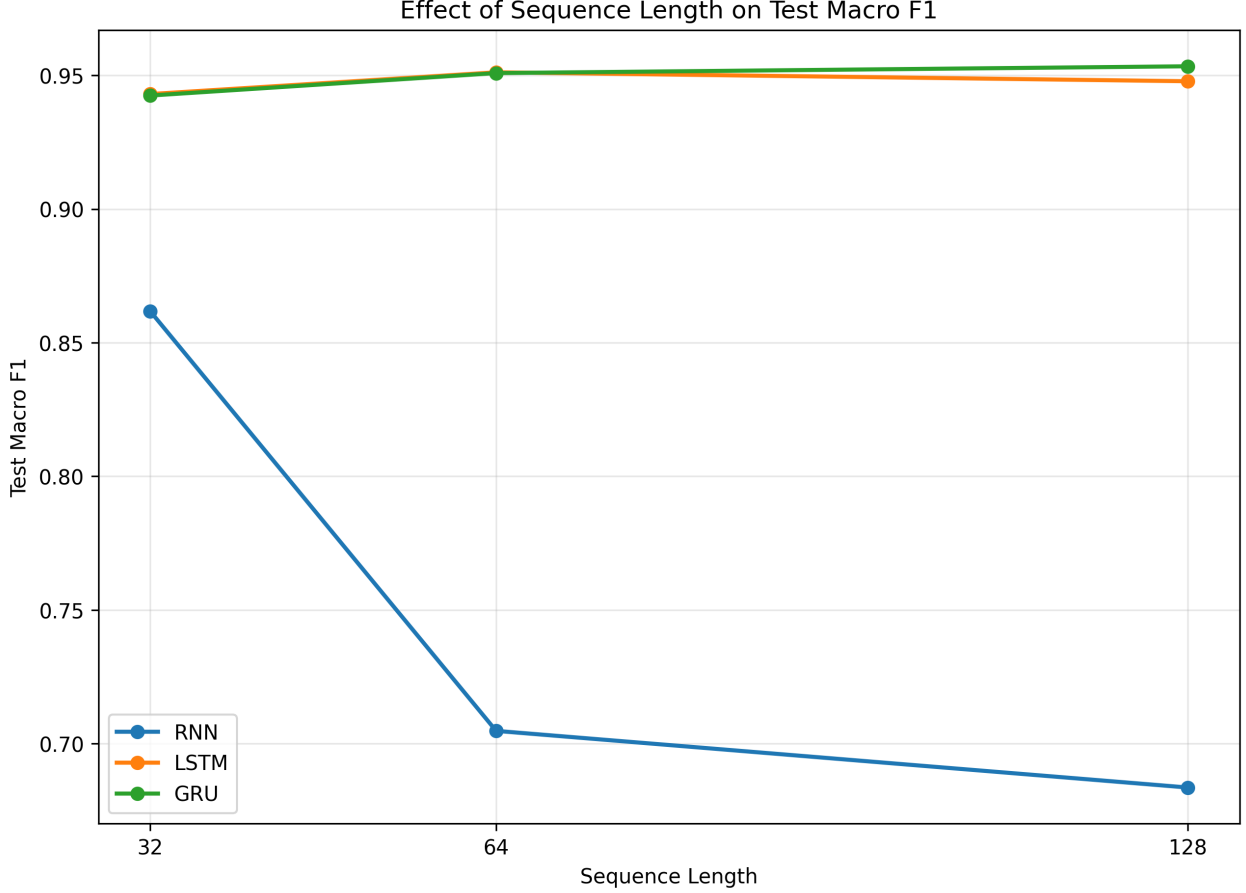


Figure 12: Macro F1 as a function of sequence length.

Inference. The Vanilla RNN macro F1 decreases as the sequence length increases in this execution. LSTM and GRU remain in the 94–95% range, showing much greater stability. Thus, the usefulness of additional temporal context depends strongly on the recurrent architecture.

14. Video Understanding Using CNN and Recurrent Networks

A small UCF101 subset containing ApplyEyeMakeup, Archery and BabyCrawling was used. Ten uniformly sampled frames were extracted from each video and resized to $224 \times 224 \times 3$.

A frozen pretrained MobileNetV2 with global-average pooling transformed each frame into a 1280-dimensional feature vector. Consequently, each video was represented by

$$X_{video} \in \mathbb{R}^{10 \times 1280}.$$

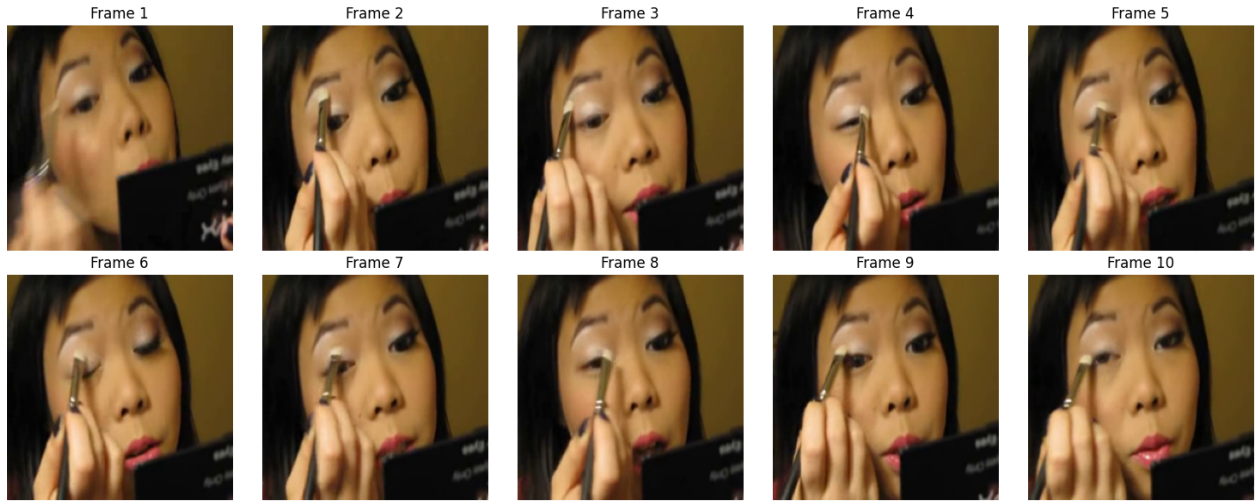


Figure 13: Ten uniformly sampled frames from a representative UCF101 video.

14.1 CNN-LSTM and CNN-GRU Models

Both models used 32 recurrent units followed by Dense(16, ReLU) and a three-class softmax output. The CNN was frozen.

Model	Accuracy	Precision	Recall	F1	Parameters	Time (s)
CNN-LSTM	100.000%	100.000%	100.000%	100.000%	168,643	5.687
CNN-GRU	94.737%	95.238%	95.238%	94.872%	126,723	7.767

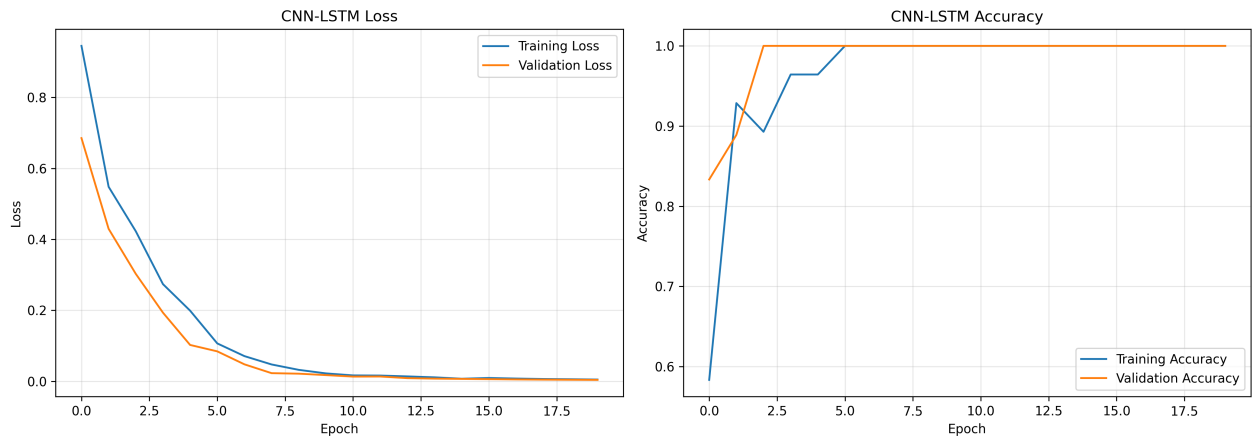


Figure 14: CNN-LSTM training and validation curves.

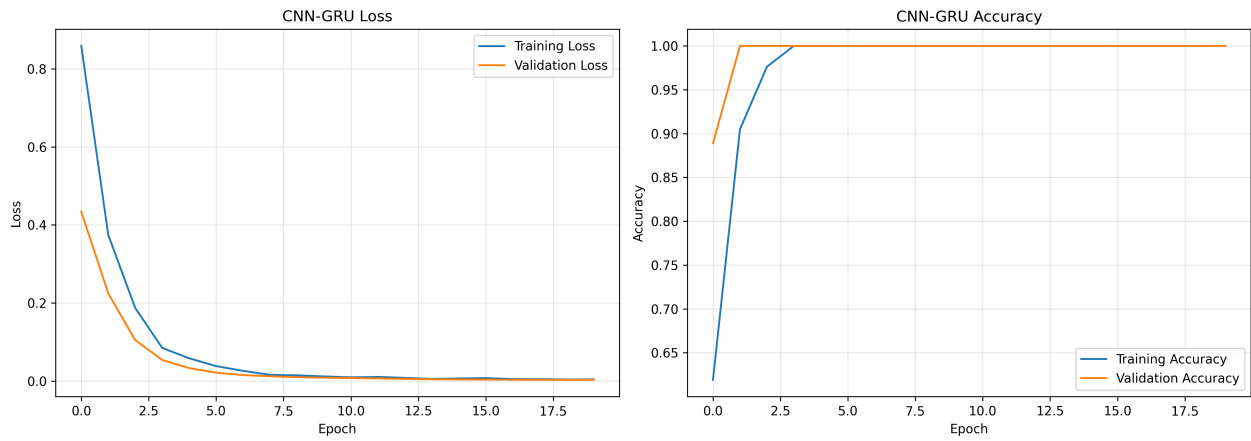


Figure 15: CNN-GRU training and validation curves.

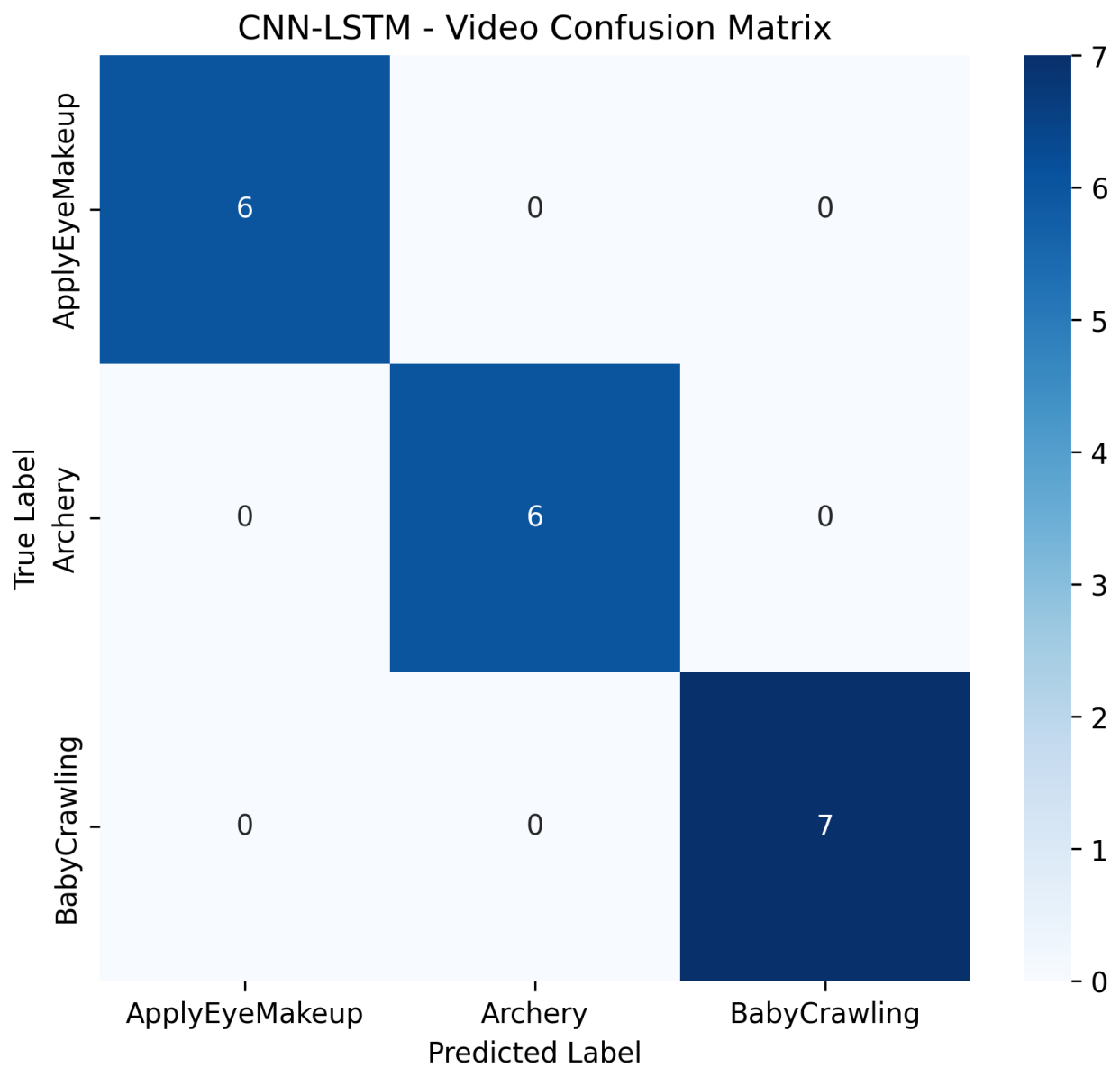


Figure 16: CNN-LSTM video confusion matrix.

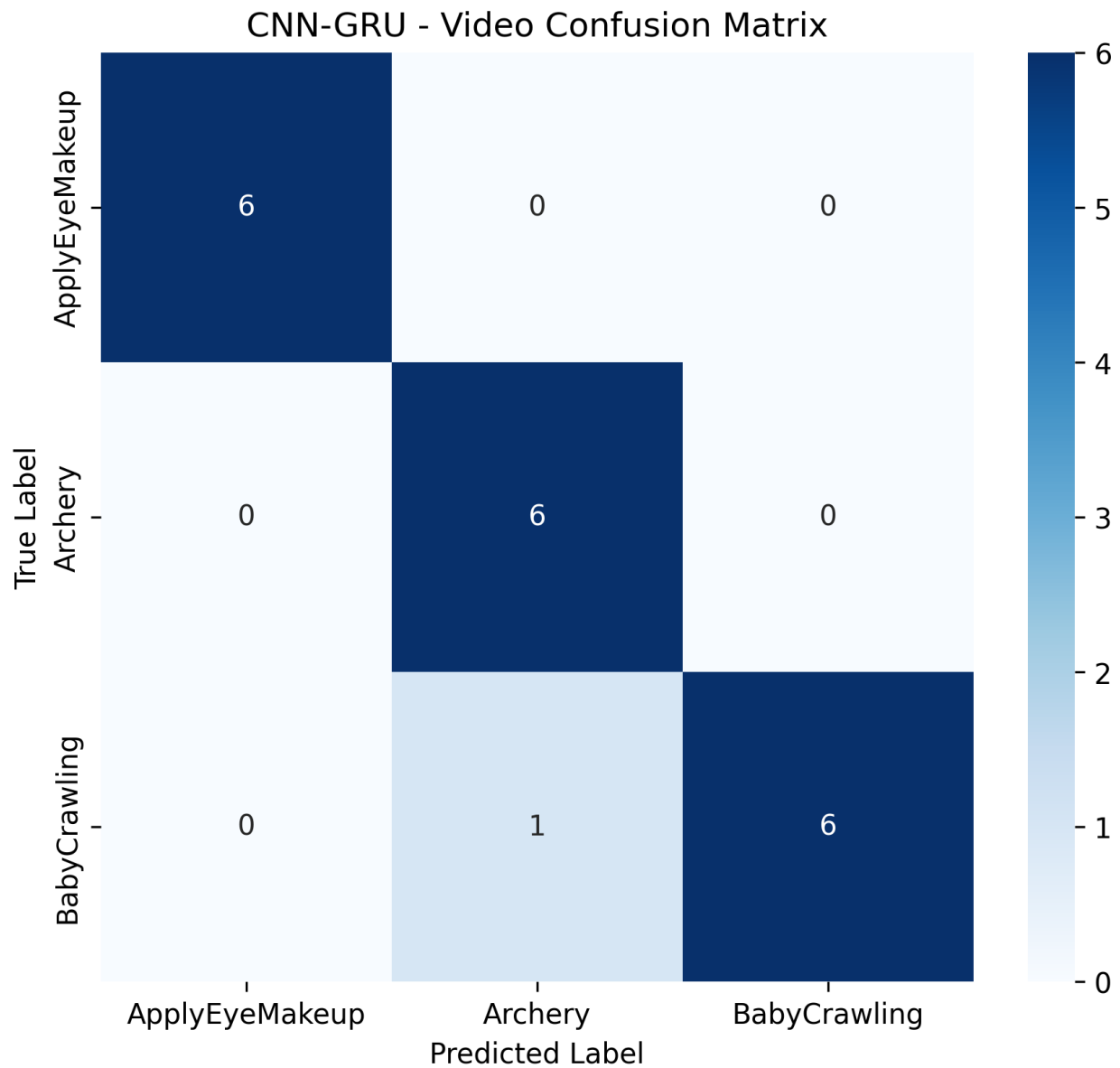


Figure 17: CNN-GRU video confusion matrix.

The CNN-LSTM confusion matrix is completely diagonal on the displayed test set: all 19 test videos are correctly classified. The CNN-GRU matrix has one BabyCrawling-to-Archery error and otherwise correct classifications. Because the test set is small, these results should be interpreted as results for the selected subset rather than as a broad estimate of UCF101 performance.

Inference. The CNN performs spatial feature extraction from each frame, while the recurrent network models temporal changes across the ordered feature sequence. This division of labor makes it possible to perform video classification without feeding raw pixels directly into the recurrent layer.

15. Sequence-to-Sequence Learning

A synthetic sequence-reversal problem was constructed. Input sequences contain integers from 1 to 9 and the target sequence is the reverse of the input. The model uses an encoder LSTM and decoder LSTM with teacher forcing during training.

For an input sequence

$$[1, 4, 7, 2],$$

the target is

$$[2, 7, 4, 1].$$

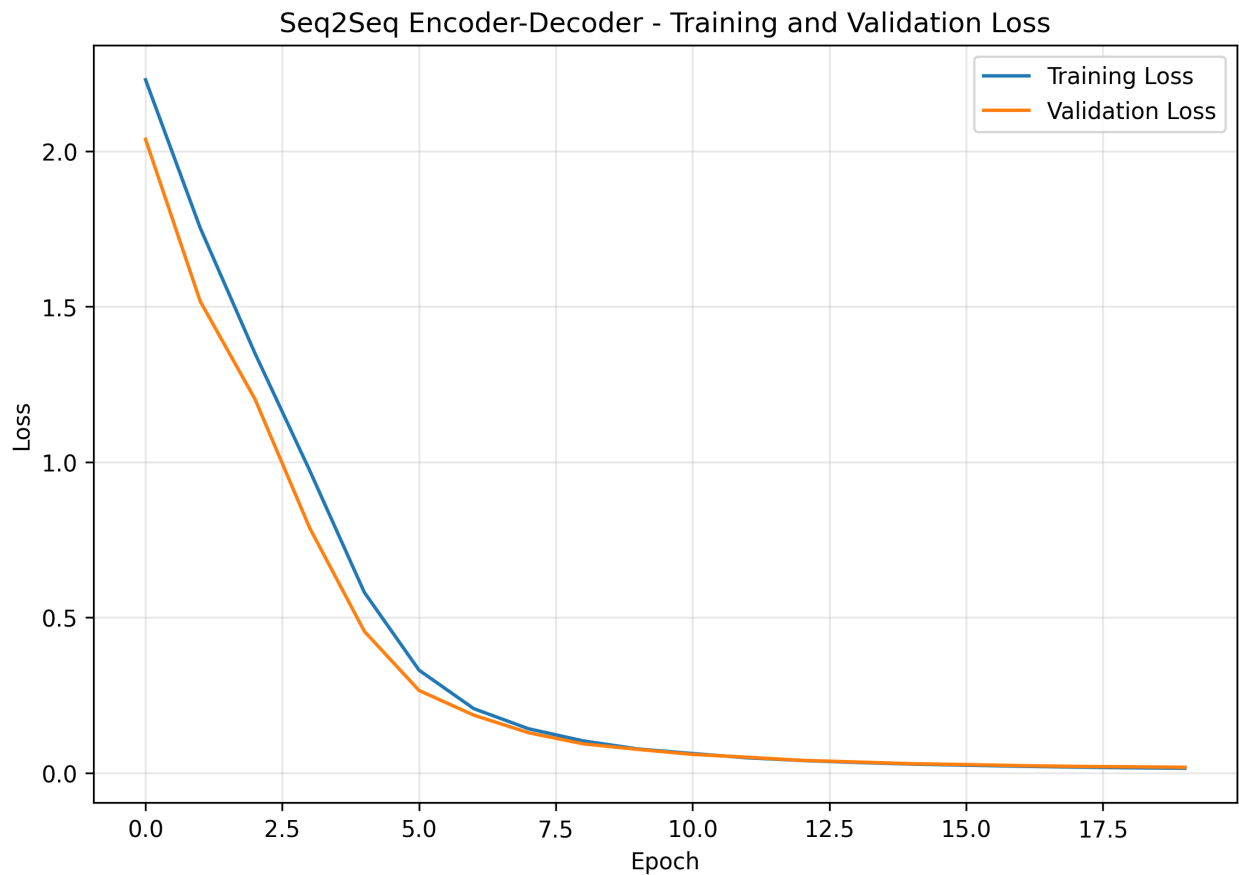


Figure 18: Sequence-to-sequence training and validation loss.

Metric	Actual result
Token Accuracy	99.9375%
Sequence Accuracy	99.7500%
Training Time	18.338 s
Trainable Parameters	50,954

Inference. Token accuracy counts individual output elements that are correct, whereas sequence accuracy requires the entire predicted sequence to be correct. The small difference between the two metrics illustrates that a few token-level errors can make an entire sequence incorrect.

Five required test examples. The laboratory specification requires at least five actual test examples showing input and predicted sequences. The exported numerical result file did not include the printed example rows, so these values are intentionally not fabricated. They should be pasted from the executed notebook output into the following table:

Example	Input Sequence	Predicted Sequence
1	Paste actual notebook output	Paste actual notebook output
2	Paste actual notebook output	Paste actual notebook output
3	Paste actual notebook output	Paste actual notebook output
4	Paste actual notebook output	Paste actual notebook output
5	Paste actual notebook output	Paste actual notebook output

16. Consolidated Results

Model	Accuracy	Precision	Recall	F1	Parameters	Time (s)
Vanilla RNN	80.000%	78.936%	79.112%	78.994%	1,974	78.606
LSTM	94.887%	95.229%	95.221%	95.225%	6,006	78.861
GRU	95.340%	95.661%	95.676%	95.659%	4,758	73.950
CNN-LSTM	94.887%	95.229%	95.221%	95.225%	6,006	78.861
CNN-GRU	95.340%	95.661%	95.676%	95.659%	4,758	73.950

17. Required Inferences

17.1 Temporal patterns. The temporal plots show that activity identity is encoded in patterns evolving across time rather than in isolated measurements.

17.2 Convergence. LSTM and GRU exhibit more stable high-performance learning behavior than the Vanilla RNN, consistent with their gated information-flow mechanisms.

17.3 Overfitting and underfitting. The curves do not show a long, persistent divergence in which training performance continuously improves while validation performance deteriorates. The experiment therefore provides stronger evidence of convergence and fluctuation than severe persistent overfitting.

17.4 Confusion-matrix errors. The main recurring error among the gated models is the SITTING-STANDING distinction, while the Vanilla RNN also has substantial confusion among walking-related classes.

17.5 Sequence length. Longer sequences strongly reduce RNN F1 in this execution, while LSTM and GRU remain comparatively stable.

17.6 Model complexity. RNN uses 1,974 parameters, GRU 4,758 and LSTM 6,006. The gated models therefore introduce additional capacity in exchange for improved temporal modeling.

17.7 CNN role. MobileNetV2 transforms each raw frame into a 1280-dimensional spatial feature vector.

17.8 Recurrent temporal modeling. LSTM/GRU receives the ordered feature vectors and learns how visual information changes across the ten sampled frames.

17.9 Token vs sequence accuracy. A sequence may contain several correct tokens but still be counted as incorrect under sequence accuracy if even one token is wrong.

18. Discussion Questions and Answers

1. **What is an RNN?** An RNN is a neural network designed for sequential data that carries a hidden state from earlier time steps to later time steps.
2. **What is the hidden state?** It is a learned representation that combines previous information with the current input.
3. **Why is sequence order important?** Temporal order can change the meaning of a sequence, so the model must preserve the ordering of observations.
4. **Feed-forward network vs RNN?** A feed-forward network has no recurrent state; an RNN connects information across time using a hidden state.
5. **What is BPTT?** Backpropagation Through Time unrolls the recurrent computation and applies gradient-based learning across the time steps.
6. **Why do gradients vanish?** Repeated multiplication by derivatives smaller than one can make gradients shrink toward zero.
7. **Why do gradients explode?** Repeated multiplication by large derivatives or transformations can make gradients grow excessively.
8. **What are long-term dependencies?** They are relationships where an earlier observation remains relevant many time steps later.
9. **What is the LSTM cell state?** It is a memory pathway through time whose information flow is regulated by gates.
10. **What are the LSTM gates?** Forget, input and output gates control what is removed, stored and exposed.
11. **What are GRU gates?** The reset gate controls previous-state influence and the update gate controls how the state is updated.
12. **LSTM vs GRU?** LSTM has a separate cell state and more gates; GRU is simpler and normally has fewer parameters for the same unit count.
13. **Why is GRU simpler?** It combines information-control functions into two main gates and does not use a separate cell state.
14. **Why use identical settings for model comparison?** It makes the comparison controlled by reducing

confounding factors.

15. **What does a confusion matrix reveal?** It shows which specific classes are confused rather than only reporting one aggregate accuracy.
16. **Why use macro F1?** Macro F1 gives equal importance to each class by averaging class-wise F1 scores.
17. **Why does sequence length matter?** It controls how much temporal context the model receives and can affect both learning and computational cost.
18. **Why use a CNN for video?** CNNs are effective at extracting spatial visual representations from individual frames.
19. **What does the CNN extract?** It learns spatial and higher-level visual patterns from each frame.
20. **What does the recurrent network learn in video?** It learns temporal relationships among the ordered frame-level feature vectors.
21. **Why arrange CNN features as a sequence?** Video frames are ordered in time, so their feature vectors must retain that order for temporal modeling.
22. **What are encoder and decoder?** The encoder converts the input sequence into a learned representation and the decoder generates the output sequence step by step.
23. **What is teacher forcing?** During training, the decoder receives the previous ground-truth target token as its previous input.
24. **Token vs sequence accuracy?** Token accuracy measures individual token correctness; sequence accuracy requires the complete sequence to be correct.
25. **Why keep the test set untouched?** Test-set use during model selection leaks evaluation information and can make final performance optimistic.

19. Conclusion / Expected Outcome

The experiment successfully demonstrated the complete sequence-learning workflow from raw temporal sensor data to recurrent classification, quantitative evaluation, sequence-length analysis, CNN-assisted video understanding and encoder-decoder sequence generation.

On UCI HAR, the Vanilla RNN achieved 80.000% test accuracy and 78.994% macro F1, while LSTM achieved 94.887% and 95.225%, and GRU achieved 95.340% and 95.659%, respectively. GRU used fewer parameters than LSTM and had the shortest recorded training time among the three recurrent models.

The video experiment showed that a frozen MobileNetV2 can provide compact frame-level spatial representations that can be modeled temporally using LSTM or GRU. The selected CNN-LSTM test set was classified with 100% accuracy, while CNN-GRU achieved 94.737% accuracy. These video results are based on a small selected subset and should be interpreted accordingly.

The sequence-to-sequence reversal model achieved 99.938% token accuracy and 99.750% sequence accuracy, demonstrating successful encoder-decoder learning.

20. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sepp Hochreiter and Jürgen Schmidhuber, "Long Short-Term Memory," *Neural Computation*, 1997.
3. Kyunghyun Cho et al., "Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation," EMNLP, 2014.
4. Anguita et al., "A Public Domain Dataset for Human Activity Recognition Using Smartphones," ESANN, 2013.
5. Khurram Soomro, Amir Roshan Zamir and Mubarak Shah, "UCF101: A Dataset of 101 Human Actions Classes From Videos in the Wild," 2012.
6. TensorFlow and Keras documentation.

All numerical results in this report are taken from the executed experiment outputs supplied with the report figures and CSV files. No model performance values were assumed.